

Calls Worth Flagging

- **The INSERT ... ON CONFLICT pattern is the correctness gate.** Reading first then inserting is a race: between read and insert, another concurrent webhook delivery can land. Atomic insert with unique-constraint-violation handling is the only safe shape.
- **Idempotency-key linkage requires the §14.7 fix to be in place.** This handler depends on the deterministic idempotency-key contract from §14.7.
- **The reconciliation job is not optional.** Without it, a handler crash mid-processing produces a permanently-stuck row that silently fails to complete. Build it in Phase 1 alongside the webhook handlers themselves.

Section 8 — RAG Service Architecture

Architectural details for the standalone RAG service: deployment, authentication, API contracts, and inter-service patterns.

8.1 Deployment

- Separate Next.js project deployed on Vercel
- Own Supabase project (separate from main app)
- Endpoint pattern: rag.ai-travelconcierge.com or similar subdomain
- Independent versioning and deployment cadence

8.2 Tenant Registry Sync

The RAG service maintains a `tenant_registry_shadow` table mirroring the main app's `tenants` table. The shadow is kept in sync via the webhook from `/api/tenant-events` (§8.7) plus a nightly reconciliation cron. See §8.3 for the full shadow-table design and the fail-closed authentication contract.

Lazy auto-registration from JWT claims (earlier spec language) is REMOVED. It created a tenant-injection vector and has been replaced by the explicit shadow-table model. If a JWT arrives for a `tenant_id` not in the shadow, the request fails with 403; the tenant lifecycle event will sync via the main app's retry queue (per §8.7a) and nightly reconcile.

8.3 Service-to-Service Authentication

All inter-service calls use JWT signed by the main app, verified by the RAG service.

- Algorithm: RS256
- TTL: 5 minutes
- Claims: `tenant_id`, `user_id` (or `service-identifier`), `persona_id`, `scope` (read/write), `jti`
- Replay protection: `jti` cache in RAG service Redis

- Signing key in environment variable; rotated every 90 days with overlap window (raised from “annually” per H-5 — RS256 keys protecting cross-service auth should rotate more frequently than app-data encryption keys)

Fail-closed contract

Every check in the JWT validation path fails CLOSED. There is no fallback or fail-open mode. Each potential failure mode and its behavior:

Failure mode	Response	Why
JWT signature invalid	401 reject	Forged or wrong key
JWT expired (>5 min old)	401 reject	Replay window exceeded
jti already seen (Redis cache)	401 reject	Replay attempted
Redis unreachable	503 reject	Cannot verify replay protection; FAIL CLOSED. Do NOT proceed with unchecked jti.
tenant_id not in registry shadow	403 reject	Tenant unknown to RAG; lazy auto-registration removed per §8.2
tenant_id in registry shadow but status != 'active'	403 reject	Tenant suspended or terminated
Any other unexpected error	500 reject	Surface for investigation; never proceed with partial validation

The contract is: if any check cannot complete, the request fails. There is no “best effort” mode.

Shadow tenant table

Per H-5 and C-7, the previous “lazy auto-registration” pattern is REMOVED. The RAG service must independently know which tenants are valid; it cannot derive that information from a JWT alone.

The RAG service maintains a `tenant_registry_shadow` table that mirrors the main app’s tenants table, kept in sync via:

1. **Webhook from main app on every tenant lifecycle event** (create, status change, terminate) — §8.7.
2. **Nightly full reconciliation cron** that compares the shadow against `/api/admin/tenants` listing from the main app. Any drift produces an alert; the reconcile process corrects it.

```
CREATE TABLE public.tenant_registry_shadow (
  tenant_id UUID PRIMARY KEY,
  status TEXT NOT NULL CHECK (status IN (
    'active', 'suspended', 'terminated',
    'pending_review', 'onboarding'
  )),
```

```

tenant_type TEXT NOT NULL,
display_name TEXT NOT NULL,
last_webhook_sync_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
last_reconcile_sync_at TIMESTAMPTZ,
source_revision INTEGER NOT NULL DEFAULT 0
);

CREATE INDEX tenant_registry_shadow_status_idx
ON public.tenant_registry_shadow(status);

```

The `source_revision` increments on every change in the main app and is shipped with the webhook payload. The RAG service uses it to detect out-of-order webhook delivery (a `source_revision < shadow.source_revision` webhook is ignored as stale).

Webhook delivery on failure

If the main app's webhook to RAG fails (network error, RAG service down), the main app:

1. Retries 3 times with exponential backoff (1s, 5s, 30s).
2. On final failure: writes to a `pending_rag_sync` table in the main app (per §8.7a) and returns success to the originating handler. The tenant lifecycle event is NOT blocked on RAG sync.
3. A periodic cron (`rag-sync-retry`, every 5 minutes) processes `pending_rag_sync` rows.
4. The nightly reconciliation handles anything still stuck.

A tenant lifecycle event that cannot reach RAG within 24 hours produces a platform admin alert. This bounds the worst-case inconsistency window.

Calls Worth Flagging

- **Redis unavailability is treated as service failure, not a bypass condition.** The alternative (allow the request when Redis is down) re-opens the replay-attack window. Operational cost: a Redis outage takes down RAG retrieval. Mitigation: Redis HA + monitoring.
- **The shadow table is part of the RAG service Supabase project, not the main app's.** Maintaining a copy is a deliberate duplication for defense in depth — the RAG service should be able to authorize requests without round-tripping to the main app.
- **90-day key rotation is a tighter cadence than the encryption key rotation (\$28.20 annual).** Different threat models: app encryption keys protect data at rest; JWT signing keys protect cross-service auth. A leaked signing key is a faster path to tenant impersonation.
- **The nightly reconcile is the safety net for the entire shadow system.** Without it, a missed webhook becomes permanent. The reconcile catches both directions of drift (extra rows in shadow, missing rows in shadow, status mismatches).

8.4 POST /api/retrieve

The primary retrieval endpoint. Called by main app on every chat turn that needs knowledge.

Request

```
{  
  "query": "What are the best balcony cabins on Symphony of the  
  Seas?",  
  "tenant_id": "uuid",  
  "user_id": "uuid",  
  "conversation_id": "uuid",  
  "persona_id": "uuid",  
  "filters": {  
    "category": "cabin_intel",  
    "cruise_line": "Royal Caribbean",  
    "ship": "Symphony of the Seas",  
    "destination": null,  
    "agent_slug": "marcus"  
  },  
  "top_k": 5,  
  "include_closed_promos_for_contact": "contact-uuid-if-applicable"  
}
```

Response

```
{  
  "chunks": [  
    {  
      "id": "uuid",  
      "content": "...",  
      "category": "cabin_intel",  
      "source_url": "...",  
      "source_domain": "royalcaribbean.com",  
      "scoring": {  
        "match_score": 0.91,  
        "authority": 0.92,  
        "authority_tier": "official",  
        "recency": 0.85,  
        "composite_confidence": 0.89  
      }  
    }  
  ]  
}
```

```

},
"metadata": {
  "ingested_at": "...",
  "expires_at": null,
  "is_promo": false
}
}
// ...more chunks
],
"retrieval_id": "uuid",
"retrieval_latency_ms": 124
}

```

8.5 POST /api/ingest

Submission endpoint. Accepts raw content for the normalization pipeline (§22). Returns queue item ID for status tracking.

8.6 POST /api/approve/tenant and /api/approve/global

Approval endpoints called by main app when a tenant admin (or platform admin for global) approves a queue item. Creates the chunk.

8.7 POST /api/tenant-events

Webhook receiver for tenant lifecycle events from main app: create, update status, terminate. Keeps tenant_registry in sync.

8.7a Main-App Retry Queue for RAG Sync

The main app retries failed webhook deliveries to the RAG service before falling back to the nightly reconcile (which is the safety net per §8.3). The retry queue is stored in pending_rag_sync table.

Schema

```

CREATE TABLE public.pending_rag_sync (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON DELETE
RESTRICT,
  event_type TEXT NOT NULL CHECK (event_type IN (
    'tenant.created', 'tenant.status_changed',
    'tenant.terminated', 'tenant.metadata_updated'
  )),
  payload JSONB NOT NULL,

```

```

source_revision INTEGER NOT NULL,
attempt_count INTEGER NOT NULL DEFAULT 0,
next_retry_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
last_attempt_at TIMESTAMPTZ,
last_error TEXT,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
delivered_at TIMESTAMPTZ
);

CREATE INDEX pending_rag_sync_ready_idx
ON public.pending_rag_sync(next_retry_at)
WHERE delivered_at IS NULL;

```

Flow

1. Tenant lifecycle event occurs in main app.
2. Handler tries direct webhook delivery to RAG service.
3. If delivery succeeds (HTTP 200): proceed normally; do NOT write to pending_rag_sync.
4. If delivery fails: write a pending_rag_sync row with next_retry_at = NOW() + 1 minute.
5. Ingest cron rag-sync-retry runs every 5 minutes:
 - Selects rows where delivered_at IS NULL AND next_retry_at <= NOW().
 - For each, attempts webhook delivery with the stored payload.
 - On success: set delivered_at = NOW(). Row is hard-deleted in nightly cleanup.
 - On failure: increment attempt_count; compute backoff; set next_retry_at. Log last_error.
6. If a row reaches attempt_count >= 10 (~20 hours of total retry time): alert platform admin.

Backoff schedule

Attempt	Delay before next retry
1 (after initial failure)	1 minute
2	5 minutes
3	15 minutes
4	30 minutes
5	1 hour
6	2 hours
7+	4 hours (capped)

Ordering and source_revision

The source_revision column (per Batch 4 B4-E5) is preserved on the retry queue. The RAG service's shadow-table sync logic uses source_revision to detect out-of-order delivery: a webhook with source_revision < shadow.source_revision is acknowledged but ignored. The retry queue does NOT need to deliver events in order.

Relationship to nightly reconcile

The retry queue handles short-term recovery (minutes to hours). The nightly reconcile (per §8.3) is the safety net for cases the retry queue cannot fix (extended RAG service downtime, handler bugs, schema

mismatches).

Calls Worth Flagging

- **pending_rag_sync rows are deleted on success.** No retention beyond active retry.
- **Idempotency on the RAG side matters.** The source_revision filter is the primary defense against duplicate processing.
- **Platform admin alert at attempt_count >= 10 is a real intervention trigger.**
- **Order independence enables aggressive parallelism.**

8.8 PII Redaction at Ingestion

Per §6.11 and §22.4: regex pre-filter for obvious patterns, then Haiku pass for subtler cases. Quarantine zero-tolerance items (passport, credit card). Redact tolerable items (names, emails, phones) with placeholders.

8.9 Scope Isolation

RAG scope is strictly two-level: global (visible to all tenants) and tenant (visible only to that tenant). There is no inheritance, no ancestor walk, no cross-tenant visibility. A tenant’s submitted content stays in that tenant unless platform admin explicitly promotes it to global per §22.8.

Section 9 — AI Personas

Six AI personas serve customers. Each has distinct expertise, voice, and visual identity. Personas are globally defined; tenants can override display name (Pro and Agency tiers) and append custom instructions (Agency only).

9.1 Persona Roster

Persona	Slug	Specialty
Marcus Cole	marcus	Caribbean / Latin America + CATCH-ALL (default routing)
Marco Bellini	marco	Mediterranean / European Rivers
Priya Sharma	priya	Luxury cruises + ship-within-a-ship (Forbes 5-star concierge background)
Captain Dave Kowalski	dave	Alaska / Adventure cruises
Maya Patel	maya	Accessible / Inclusive cruises (OT background, wheelchair user)

9.2 Persona Selection

- Customer can manually pick a persona from ‘Meet our advisors’ page
- AI router selects appropriate persona based on customer’s initial query topic (defaults to Marcus if unclear)
- AI may suggest mid-conversation handoff: ‘This is more in Maya’s wheelhouse — should I get her involved?’
- Customer can override and stay with current persona

9.3 System Prompt Architecture

Each persona’s system prompt is built from three layers, in order:

- Base persona block — character, voice, expertise, visual identity, base instructions
- Platform constraints — disclosure rules, prohibited topics (medical/legal/financial advice), escalation triggers
- Tenant addendum (Agency tier only) — appended after platform constraints; Haiku-screened for safety before save

The full prompt is cached (Anthropic prompt caching) to reduce inference cost on subsequent turns.

9.4 Persona Base Block Structure

Each base block establishes:

- Character: name, background, voice (warm professional / direct / playful)
- Expertise area: regional and topical focus
- Anti-instructions: never claim to be human, never give medical/legal/financial advice, never make commitments on behalf of host
- Tone calibration placeholder: filled at runtime per §24 tone matching
- Disclosure pattern: how to introduce self when asked

9.5 Tenant Persona Overrides

```
CREATE TABLE public.tenant_persona_overrides (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id UUID NOT NULL REFERENCES public.tenants(id),  
  persona_slug TEXT NOT NULL,
```



```
display_name_override TEXT, - Pro/Agency tier
system_prompt_addendum TEXT, - Agency tier only
is_disabled BOOLEAN DEFAULT FALSE, - any tier
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE (tenant_id, persona_slug)
);
```

9.6 Tool Use

Personas have access to tools via Anthropic’s tool use API:

- `search_host_inventory` — query host adapter for cruise availability
- `get_customer_context` — fetch CRM + memory for current customer
- `generate_quote` — create a quote line item set
- `collect_booking_details` — initiate booking flow with pre-fills
- `escalate_to_human` — surface to tenant’s supervisor
- `update_memory` — write to customer memories (with consent check)

9.7 Disclosure Rules

AI personas always disclose AI nature when asked. Persistent UI disclosure (§24) says ‘AI-assisted chat’. State-specific AI disclosure laws (California, Illinois, Utah, New York) covered by the AI Liability Disclaimer (§17.6) that every user accepts.

9.8 In-Conversation Persona Switching

Customer → Marcus (“Hi, I want a Caribbean cruise”)

|

| (customer mentions accessibility needs)

|

|→ Marcus suggests handoff: "Should I get Maya involved?"

|

▼ (customer confirms)

|

Customer → Maya (“Hi, I’m Maya, taking over from Marcus...”)

[previous conversation summary preserved as context]

9.9 Streaming via SSE

Chat responses stream via Server-Sent Events. Client subscribes to /api/chat with the message; server streams response chunks. Reconnect supported via Last-Event-ID header.

9.10 AI Mode (Tenant-Level)

Tenants control how AI participates in customer conversations via the ai_mode setting on their tenant configuration.

Modes

Mode	Customer-facing chat	AI drafts available to tenant agent?	Other AI processes
autonomous	AI responds directly	N/A (responses go direct)	All run
draft_only	Tenant agent reviews every AI draft before sending	Yes — required path	All run
disabled	Tenant agent handles all chat manually; no AI in customer chat	No	Some still run — see §9.10.1

9.10.1 What disabled actually disables

ai_mode = 'disabled' means **no customer-facing AI chat responses**. It does NOT mean zero AI processing. The following continue to run regardless of ai_mode:

AI process	Runs when ai_mode = disabled?	Why
Customer chat persona response	NO	The point of disabled mode
Customer chat persona draft for agent review	NO	Same
Memory extraction from conversations (§11.2)	YES	Conversations still happen; memory captures customer facts the agent typed
Persona addendum Haiku screening (§16.6)	YES	Tenant may still configure personas for future use
RAG ingestion pipeline normalization (§22)	YES	Tenant may still submit content for their corpus
Abuse signal screening on RAG	YES	Platform protection regardless of tenant

submissions (§22.4)		mode
Forum moderation (§19.3)	YES	Tenant participation in group forums
Supervisor preflight checks	NO (no AI response to check)	N/A
Pre-cruise email rendering with personalization (§23.4)	YES (if email send enabled)	Emails are platform-orchestrated, not customer-chat

AI Mode is a customer-chat-scoped control, not a platform-wide AI off switch.

9.10.2 Tenant-facing UI requirements

The `ai_mode` configuration screen MUST:

- Show the three modes with the behavior description from the §9.10 table above.
- Show an expandable “What does disabled NOT cover?” section reproducing the §9.10.1 table.
- Show estimated monthly AI cost ranges for each mode at the tenant’s current usage pattern (per §27.12 cost attribution).
- For disabled specifically: show a callout “If you need TRULY zero AI processing — including memory extraction and persona screening — see Background AI Settings below.”

9.10.3 Background AI Settings (separate toggle)

For tenants who want zero AI processing of any kind, a separate “Background AI” toggle exists. Default: ON. When OFF:

- Memory extraction does not run.
- Persona addendums cannot be saved (Haiku screening required).
- RAG ingestion content normalization runs with reduced AI (basic rule-based normalization only; PII detection still runs via the zero-tolerance pipeline per §22.4).
- Pre-cruise email rendering uses template-only personalization (no AI-generated narrative).
- Forum moderation falls back to coordinator-only (no Haiku screening — coordinator must approve every message).

Schema addition:

```
ALTER TABLE public.tenants
ADD COLUMN background_ai_enabled BOOLEAN NOT NULL DEFAULT TRUE;
```

Tenant UI banner when Background AI is OFF:

“Background AI is OFF. Some platform features have reduced functionality:

- Customer memory is not auto-extracted from conversations.
- You cannot configure persona positioning (requires AI screening).
- Pre-cruise emails use generic templates only.
- Group forum moderation requires manual coordinator approval for every message.

[Turn Background AI back on] [Learn more]”

This is a niche setting. Most tenants will leave it ON. The toggle exists for tenants with strict compliance requirements who need to assert “no AI processing of customer data at all.”

9.10.4 The trap §A.13 warned about

§A.13’s anti-pattern note (“letting `ai_mode = 'disabled'` mean ‘no AI calls ever’”) is addressed by §9.10.1 above. The §9.10 configuration screen makes the distinction explicit so tenants choosing disabled are not surprised by AI-related charges on their bill.

Calls Worth Flagging

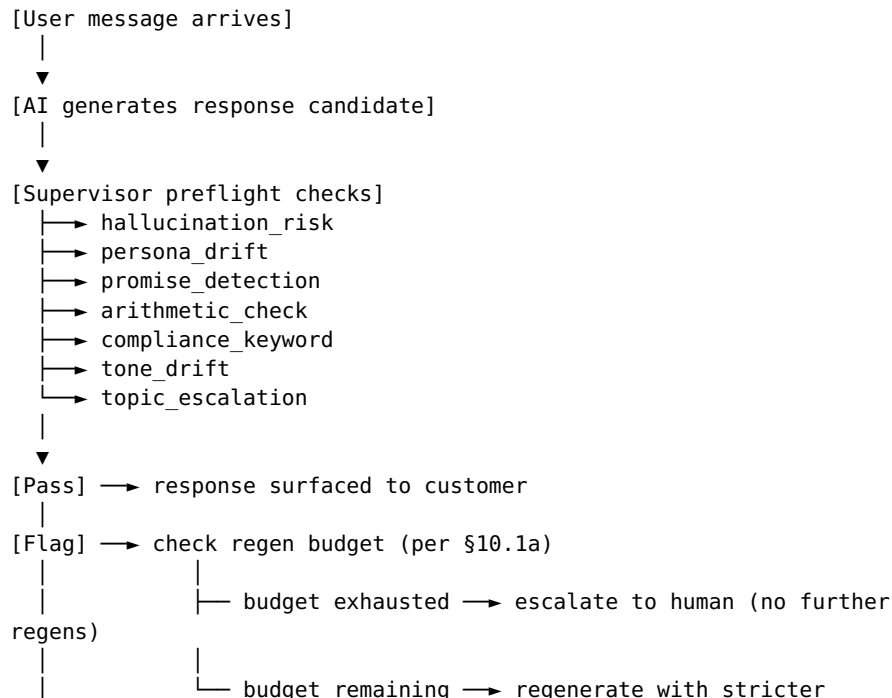
- **Most tenants who choose disabled actually want `draft_only`.** Tenant onboarding (§15) should ask about the underlying goal.
- **The Background AI toggle defaults ON.** Off should be a conscious decision.
- **Estimated monthly cost ranges in the UI require §27.12 cost attribution data.** For new tenants, “varies based on usage” is acceptable.

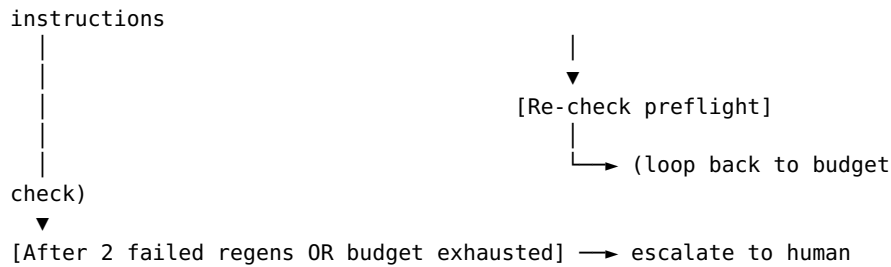
NOTE: Avatar art is user-provided AI-generated images. Asset delivery deferred to implementation.

Section 10 — AI Supervisor

The supervisor is a safety and quality layer that runs around every AI response. It performs preflight checks before responses are surfaced to customers, handles escalations, and feeds into platform-wide monitoring.

10.1 Architecture





10.1a Per-Conversation Regen Budget

A bad persona prompt, a prompt-injection attack, or genuinely challenging content can produce repeated regen cycles. Each regen is a full Sonnet call. Without a cap, a single afternoon of regen spirals can consume a tenant’s entire monthly AI budget — and \$27’s monthly thresholds catch it too late.

Budget structure

Each conversations row tracks the cumulative regen activity:

```

ALTER TABLE public.conversations
  ADD COLUMN regen_tokens_consumed INTEGER NOT NULL DEFAULT 0,
  ADD COLUMN regen_count_total INTEGER NOT NULL DEFAULT 0,
  ADD COLUMN regen_budget_exhausted_at TIMESTAMPTZ;

```

The supervisor’s regen loop reads two thresholds (configurable per \$28.X — env vars below):

- SUPERVISOR_REGEN_MAX_PER_CONVERSATION (default: 6) — absolute cap on regen attempts for a single conversation across its lifetime. Independent of message count.
- SUPERVISOR_REGEN_MAX_TOKENS_PER_CONVERSATION (default: 25000) — cumulative token spend cap on regen attempts. Sonnet at ~\$3/MTok input + \$15/MTok output makes 25K tokens roughly \$0.25 — a reasonable per-conversation ceiling.

EITHER threshold trips the exhaustion state. Once exhausted:

- The current message gets one final escalation-to-human (no further regens; the in-flight regen attempt completes and is delivered or the existing response is surfaced with a warning marker).
- regen_budget_exhausted_at is set on the conversation row.
- ALL subsequent messages in this conversation skip the regen loop entirely — supervisor still preflights, but flag → direct escalation, no regeneration attempts.
- Tenant is alerted (\$10.5 supervisor dashboard shows a new “Exhausted regen budget” surface).

Budget recovery

Per-conversation exhaustion does not auto-reset. The conversation is flagged for human attention until either:

- A human agent intervenes and resets the budget manually (sets regen_budget_exhausted_at = NULL, increments regen_count_total by 0 — explicit no-op so audit log records the reset).
- The conversation is closed (status = 'closed'). A new conversation

starts fresh.

Why per-conversation, not per-tenant or per-hour

- Per-tenant: §27.4 already does this at monthly granularity. Too coarse to catch spirals within a single afternoon.
- Per-hour-globally: rate-limiting AI calls at the tenant level affects ALL conversations including healthy ones; pathological conversations need to fail without dragging down everyone.
- Per-conversation: the scope of the problem. A pathological conversation gets quarantined; other conversations in the same tenant continue normally.

Defense layering

The per-conversation cap is the FIRST line. Defense in depth:

Layer	Trigger	Effect
Per-conversation cap (§10.1a)	6 regens OR 25K tokens in one conversation	Conversation quarantined, escalates to human
Per-tenant abuse monitoring (§27)	Monthly thresholds	Throttle / hard limit per tier
Global kill switch (§10.6)	Platform-wide misbehavior	All AI paused

Calls Worth Flagging

- **Defaults are deliberately conservative.** 6 regens / 25K tokens per conversation is a ceiling, not a target. Tune downward in Phase 1 if observation shows most healthy conversations need fewer regens.
- **Exhaustion is sticky on the conversation.** A conversation that hits the budget stays human-escalated even after a successful human handoff back to AI, until a human explicitly resets it. This avoids re-entering the spiral.
- **The token cap protects against a “few small regens that add up” attack.** A pathological prompt that produces 4 regens each with 6K tokens hits the token cap before the regen count cap. Both bounds together cover the failure space.

10.2 Preflight Checks

Check	What it does	Action on flag
hallucination_risk	Extract factual claims, verify against retrieved chunks	Regenerate with grounding
persona_drift	AI staying in character	Regenerate
promise_detection	Catches ‘guaranteed’, ‘I assure you’, etc.	Rewrite to hedged language
arithmetic_check	Validates any math in response Detects medical,	Regenerate

compliance_keyword	legal, financial advice patterns	Escalate
tone_drift (heuristic)	Tone-topic mismatch, excessive profanity in non-opted-in tenants, off-brand sycophancy	Regenerate or flag
tone_drift (lexical)	Deterministic match against slur and hate-speech deny-list per §24.5	Regenerate; auto-escalate after 3 consecutive matches
topic_escalation	Topics customer or tenant flagged for human	Defer on flagged topics

10.3 Topic-Level Escalation

Unlike conversation-level escalation, the supervisor tracks escalation at the TOPIC level. A customer may have an active escalation about a refund (handled by human) while continuing to chat about an unrelated future trip (AI continues to handle).

```
CREATE TABLE public.escalation_topics (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id),
  conversation_id UUID NOT NULL REFERENCES public.conversations(id),
  user_id UUID REFERENCES public.users(id),
  topic_summary TEXT NOT NULL,
  topic_tags TEXT[],
  status TEXT NOT NULL CHECK (status IN ('open', 'in_progress', 'resolved', 'closed')),
  initiated_by TEXT, - 'supervisor', 'customer', 'agent'
  initiated_reason TEXT,
  assigned_agent_id UUID REFERENCES public.users(id),
  opened_at TIMESTAMPTZ DEFAULT NOW(),
  resolved_at TIMESTAMPTZ,
  resolution_notes TEXT
);
```

When uncertain whether a new message relates to an open escalation, the AI asks for clarification rather than guessing.

10.4 Supervisor Findings (per message)

Every preflight check produces a finding (pass or flag). Findings stored in `messages.supervisor_findings` JSONB:

```
{
  "checks_run":
  ["hallucination_risk", "persona_drift", "promise_detection", "arithmetic_
  check", "compliance_keyword", "tone_drift", "topic_escalation"],
  "findings": [
    { "check": "hallucination_risk", "severity": "info", "details": "All factual
    claims supported" },
    { "check": "promise_detection", "severity": "warning", "details": "Soft-
    rewrote 'always available' to 'typically available'" },
  ],
  "regen_count": 0,
  "final_action": "allow"
}
```

10.5 Supervisor Dashboard

- Open topic-level escalations (assigned + unassigned)
- Recent flagged messages by check type
- Per-persona metrics: response quality, thumbs-down rate, regen rate
- Per-tenant aggregate metrics (visible to platform admin)
- Alert feed (priority-sorted)

10.5a Supervisor Sampling and Review

The supervisor pipeline per §10.1 has three outcomes per message: **pass**, **regenerate** (then re-evaluate), and **escalate**. The “pass” outcome includes a subset where supervisor findings include severity: warning items that were auto-rewritten or auto-corrected. These borderline passes are the highest-value source for tuning the supervisor itself — cases where a check fired, the system self-recovered, but a human eye might catch a residual issue the checks missed.

This subsection defines the sampling and review surface.

What gets sampled

Four sampling rates, applied per message at supervisor completion:

Outcome class	Default sample rate	Configurable
Pass with no findings	1% (<code>supervisor_sample_rate_clean_pass</code>)	Yes

Pass with warning findings (auto-rewritten)	10% (supervisor_sample_rate_warning_pass)	Yes
Regen attempted (regardless of final outcome)	25% (supervisor_sample_rate_regen)	Yes
Escalate to human	100%	No — every escalation is reviewable already via §10.5 dashboard

Sampled messages go into a `supervisor_review_queue` table. Random per-message (not per-conversation, not per-tenant).

Queue surface

`/admin/supervisor/review-queue` displays the queue with these tabs:

Tab	Filter	Action shape
Recent (default)	All sampled messages from last 7 days	Mark as “OK” / “issue spotted”
Warning passes	Only severity: warning items	Same
Regens	All sampled regens	Same
Disputed	Items previously marked “issue spotted”	Adjudicate / annotate / close

Each item shows: original AI response, supervisor findings, conversation context (previous N messages), cited RAG chunks if applicable, customer’s subsequent thumbs reaction if any. Reviewer marks “OK” (most cases, fast) or “issue spotted” (flagged for engineering attention with a free-text reason).

What “issue spotted” triggers

- Engineering team alerted via existing audit-log + Sentry pipeline.
- Item moves to “Disputed” tab.
- Pattern surfaces in §10.7 supervisor performance metrics.
- If 3+ items flagged “issue spotted” within a 7-day window cite similar supervisor finding types, an aggregated issue is auto-created for engineering with the pattern summary.

Why this isn’t the §22.7 review queue

§22.7 reviews RAG content (chunks). This reviews supervisor verdicts on AI responses. Different domain expertise, different queue, different SLA.

Schema

```

CREATE TABLE public.supervisor_review_queue (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id),
  message_id UUID NOT NULL REFERENCES public.messages(id),
  conversation_id UUID NOT NULL REFERENCES public.conversations(id),
  sample_category TEXT NOT NULL CHECK (sample_category IN (
    'clean_pass', 'warning_pass', 'regen_attempted', 'escalation'
  )),
  supervisor_findings_snapshot JSONB NOT NULL,
  conversation_context_snapshot JSONB,
  review_status TEXT NOT NULL CHECK (review_status IN (
    'pending', 'reviewed_ok', 'reviewed_issue', 'disputed_resolved'
  )) DEFAULT 'pending',
  reviewed_by_user_id UUID REFERENCES public.users(id),
  reviewed_at TIMESTAMPTZ,
  review_notes TEXT,
  sampled_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  purge_after TIMESTAMPTZ NOT NULL DEFAULT (NOW() + INTERVAL '90
days')
);

CREATE INDEX supervisor_review_queue_status_idx
ON public.supervisor_review_queue(review_status, sampled_at DESC)
WHERE review_status = 'pending';

```

Configurable knobs (in platform_settings)

Setting	Default	Range
supervisor_sample_rate_clean_pass	0.01	0.0-1.0
supervisor_sample_rate_warning_pass	0.10	0.0-1.0
supervisor_sample_rate_regen	0.25	0.0-1.0
supervisor_review_retention_days	90	30-365

Calls Worth Flagging

- **Sampling is at supervisor-completion time, not lazily.** Atomic snapshot prevents “data I wanted to review changed before I got to it.”
- **Reviewer notes drive pattern detection.** The aggregated-issue trigger (3+ items in 7 days) is where the queue produces engineering signal.
- **Defaults err generous for launch.** Tune downward once queue’s signal-to-noise is understood.
- **Reviewer authority:** any platform admin can review. Marking “issue spotted” without a reason is allowed (required only if it’s the third item in a 7-day pattern).

10.6 Kill Switch

If AI behavior becomes problematic platform-wide (model misbehavior, prompt injection campaign, etc.), platform admin can pause AI responses globally or per-tenant. Activated AI shows: ‘Our AI is taking a brief break. A human will be in touch shortly.’

10.7 Supervisor Performance Metrics

- Hallucination check fire rate
- Block-after-regen rate
- Compliance escalation rate
- Thumbs-down rate per persona, per topic
- Tone-drift detection rate
- Average AI response latency (with supervisor overhead)

Section 11 — Customer Memory

Memory gives the AI continuity across conversations. A returning customer doesn't have to re-explain their travel preferences, family composition, or accessibility needs. Memory is tenant-scoped, customer-controlled, and respects opt-out.

11.1 Memory Storage

```
CREATE TABLE public.customer_memories (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id),
  user_id UUID NOT NULL REFERENCES public.users(id),
  contact_id UUID REFERENCES public.contacts(id),
  - Structured data
  preferences JSONB, - cruise lines, ports, cabin types, dining
  travel_history JSONB, - past trips taken
  family_composition JSONB, - partners, children (DOBs not ages),
  notable companions
  accessibility_needs TEXT,
  dietary_restrictions TEXT,
  loyalty_programs JSONB,
  important_dates JSONB, - anniversaries, birthdays
  notes_freeform TEXT,
  - Rapport (per §24 tone matching)
  rapport_tone_level INTEGER DEFAULT 2 CHECK (rapport_tone_level
  BETWEEN 1 AND 5),
  rapport_signals JSONB,
  rapport_level_set_at TIMESTAMPTZ,
  - Lifecycle
  last_extracted_at TIMESTAMPTZ,
```

```

conversation_count INTEGER DEFAULT 0,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE (tenant_id, user_id)
);

```

11.1.1 Scope discipline

The `UNIQUE (tenant_id, user_id)` constraint is the schema-level guarantee of memory isolation: a single user has exactly one memory row per tenant. Cross-tenant leak (a fact from tenant A's conversation written to tenant B's memory row) is impossible at the constraint level — the wrong `tenant_id` would produce a violation or an `INSERT` into a different row.

But the constraint only catches scope violations at write time. Read and update paths are protected by:

- RLS policies per §5.1 (`auth_user_in_tenant + tenant_is_active` for writes).
- The `tenantClient` proxy per §5.4.3 (auto-filters every query by `tenant_id`).
- The extraction job's explicit scope contract per §11.2.2.

All three together. Removing any one is a regression.

11.2 Memory Extraction

Memory extraction is the Inngest-driven background job that updates `customer_memories` from conversation content. The job's correctness depends on strict tenant scoping — a bug in scope handling produces the worst-case privacy bug shape (one tenant's customer facts written to another tenant's memory row). This subsection specifies the scoping contract and the debounce behavior.

11.2.1 Trigger conditions

Extraction is enqueued when EITHER:

- The conversation reaches a multiple of 10 messages.
- The conversation enters a `closed` or `abandoned` state.

Enqueuing produces an Inngest event with `tenant_id`, `conversation_id`, `user_id` populated from the conversation row at enqueue time. The event's `tenant_id` is the authoritative tenant scope for the extraction — NOT the user's "current" tenant or the calling request's tenant.

11.2.2 The mandatory scope contract

Extraction runs inside `withPlatformAdminAudit` is INCORRECT — it is a tenant-scoped operation, not a platform-admin one. The correct pattern:

```

export const extractMemory = inngest.createFunction(
  { id: 'customer-memory-extraction' },
  { event: 'conversation.memory_extract_requested' },

```

```

    async ({ event, step }) => {
      // tenant_id is sourced ONLY from the event payload.
      const ctx = await tenantContextFromInngestEvent(event);
      const db = tenantClient(ctx);

      // Every query MUST go through `db`. The proxy auto-injects
      // `tenant_id = ctx.tenant_id`. No raw service-role calls
      // by the lint rule in §26.3a.
      const conversation = await step.run('fetch-conversation', async
allowed
() => {
      return db
        .from('conversations')
        .select('id, user_id, status')
        .eq('id', event.data.conversation_id)
        .single();
    });

    // CRITICAL: assert the conversation's tenant matches the
context.
    // The query above is already auto-filtered by tenant_id, so a
    // mismatched conversation_id would return zero rows (not a row
    // from another tenant). The assertion is defense in depth.
    if (!conversation.data || conversation.data.user_id !==
event.data.user_id) {
      throw new Error('Conversation/user/tenant mismatch in
extraction');
    }

    // ... rest of extraction ...

    // Write goes through `db` too – auto-scoped to tenant_id.
    await step.run('update-memory', async () => {
      return db
        .from('customer_memories')
        .upsert({
          tenant_id: ctx.tenant_id,
          user_id: event.data.user_id,
          // ... fields ...
        });
    });
  },
);

```

What this gets us:

- **Single source of tenant_id:** the event payload. Not the user's default tenant, not the calling request's host header, not a derived field.
- **Auto-scoping at the query layer.** Even if a developer writes a bare `db.from('conversations').select(...)` without an explicit `tenant_id` filter, the `tenantClient` proxy from §5.4.3 adds the filter. A cross-tenant read returns zero rows, not someone else's data.
- **Defense-in-depth assertion.** The handler also explicitly verifies the conversation row's `user_id` matches the event's `user_id`. Belt-and-suspenders against a future change to the `tenantClient` proxy.
- **Inngest event id audit trail.** Every extraction run is traceable to the event that triggered it. Forensic investigations can ask “which extraction job processed conversation X?” and answer it from the audit log via §26.5.

11.2.3 Debounce

A customer who sends 100 messages in 5 minutes would otherwise trigger 10 extraction jobs. Each job is a Haiku call. Debounce coalesces these:

- The Inngest event handler checks `customer_memories.last_extracted_at` for the `(tenant_id, user_id)` row.
- If `last_extracted_at` is within the last `MEMORY_EXTRACTION_DEBOUNCE_SECONDS` (default 120 — env var per §28.16), the handler exits without running extraction. The new state is processed by the next natural trigger.
- This is rate-limit-by-design, not by-default — implementations must enforce it. The lint rule in §30.8 verifies that the extraction function reads `last_extracted_at` before issuing a Haiku call.

11.2.4 Optimistic locking on memory merge

Multiple extraction jobs can race for the same `(tenant_id, user_id)` row (rare with debounce in place, possible during the debounce window). The merge step uses optimistic locking:

```
// Read current memory with its updated_at timestamp.
const current = await db
  .from('customer_memories')
  .select('*', updated_at')
  .eq('user_id', event.data.user_id)
  .single();

// Merge extracted facts into the current memory JSONB structures.
const merged = mergeMemory(current.data, extracted);

// Conditional update: only commit if updated_at hasn't changed.
const result = await db
  .from('customer_memories')
  .update({ ...merged, updated_at: new Date().toISOString() })
  .eq('user_id', event.data.user_id)
  .eq('updated_at', current.data.updated_at)
  .select();

if (!result.data || result.data.length === 0) {
  // Another extraction job committed between our read and write.
  // Re-enqueue with a small delay.
  await inngest.send({
    name: 'conversation.memory_extract_requested',
    data: event.data,
    ts: Date.now() + 5000,
  });
  return;
}
```

The optimistic-lock retry uses Inngest's natural retry/event-send mechanism rather than a tight loop — keeps the function simple and plays well with Inngest's observability.

11.2.5 Process

- Take the last N messages from the conversation (N = 50 default; configurable via env per §28.16).
- Plus current `customer_memories` row content for the `(tenant_id,`

- user_id).
- Haiku prompt: “Extract any new facts or updates about this customer; return structured JSON.”
- Result merged with existing memory using the optimistic-locking pattern in §11.2.4.

11.2.6 Calls Worth Flagging

- **tenant_id source must be event payload, never derived.** A helpful refactor that derives tenant_id from the conversation’s user_id lookup is a bug — it removes the explicit “what tenant is this extraction for” decision from the path. The Inngeist event is the authoritative scope.
- **The proxy auto-filter is the primary defense; the assertion in the handler is defense-in-depth.** Both layers matter. If the proxy is ever bypassed (a bug in tenantClient, a future refactor that uses raw service role), the handler-level assertion catches it.
- **Debounce trades freshness for cost.** A customer sending a flurry of messages won’t have all those facts in memory immediately. For pre-cruise email personalization, the next natural extraction (conversation close, next 10-message boundary) catches up. Acceptable trade-off; document in tenant-facing copy where memory freshness matters.
- **Optimistic locking with re-enqueue is preferable to row-level DB locks** for this workload. Locks held during a Haiku call (several seconds) are too long; optimistic-lock-and-retry handles contention cleanly.

11.3 Customer-Facing Memory Controls

Customer can view, edit, and delete their memory at /settings/memory. Memory respects:

- memory_opt_out flag on users — disables memory extraction entirely for that customer
- Individual field editing — customer can correct or remove specific facts
- Full memory deletion — clears all memory for this tenant
- Per-tenant isolation — memory in tenant A invisible to tenant B

11.4 Memory in Prompts

Persona system prompt receives a ‘CUSTOMER CONTEXT’ block populated from memory:

CUSTOMER CONTEXT:

Name: Sarah Johnson

Preferences:

- Prefers balcony cabins
- Cruise lines tried: Royal Caribbean (twice), Princess (once)
- Favorite destinations: Caribbean, especially Eastern itineraries

Family: Husband Mike (DOB 1985-03-15), 2 children (DOBs 2014-05-22, 2017-09-08)

Accessibility: None noted

Dietary: Vegetarian

Loyalty: Royal Caribbean Crown & Anchor (Diamond), Princess Captain’s Circle (Gold)

Important dates: Wedding anniversary April 15

Conversation history: 8 prior conversations, 1 booking completed (Caribbean 7-night, 2024)

Rapport tone level: 3 (casual)

11.5 DOBs Not Ages

Memory and CRM store dates of birth, never ages. Ages are computed at sailing date for booking validation (cruise lines have age rules for kids, drinking, etc.). The DOB lifecycle for estimated entries is specified below to prevent old estimates feeding into personalization indefinitely.

Storage shape

```
{
  "first_name": "Mike",
  "date_of_birth": "1985-01-01",
  "date_of_birth_is_estimated": true,
  "estimation_basis": "customer said 'about 40' on 2025-03-15",
  "estimation_recorded_at": "2025-03-15T14:23:00Z",
  "estimation_last_reprompt_at": null
}
```

Visual marker for estimated DOBs

Every UI rendering of an estimated DOB MUST include a visual marker:

Surface	Rendering
CRM contact detail view	“(estimated, click to confirm)” link next to the DOB field
Booking submission gate (\$20)	“We need to confirm Mike’s exact birthday before submitting — please ask Mike to verify”
Pre-cruise emails (\$23)	DOB-derived content (e.g., “celebrating Mike’s 40th”) is SUPPRESSED for estimated DOBs
Quote PDFs	Estimated DOBs not rendered at all
Customer-facing memory view (\$11.3)	“(approx)” badge next to the DOB
Agent conversation context (\$9)	Persona instructed to never assert facts about birthdays for estimated DOBs

The principle: **estimated DOBs participate in eligibility decisions but NOT in personalization content.** Saying “celebrating Mike’s milestone birthday!” when Mike’s DOB is estimated risks an awkward customer experience if the estimate was off.

Aging-out and re-prompting

If an estimated DOB is:

- Older than 365 days from `estimation_recorded_at`, AND
- Never confirmed (no booking submitted that locked in the actual DOB via §20’s confirmation gate), AND
- The associated customer is still in active conversation with the tenant (any message in the last 90 days),

then on the customer’s next conversation turn, the AI is prompted (via persona system prompt augmentation) to gently re-confirm:

“I have a quick reminder — when we first chatted, you mentioned Mike was around 40. For trip planning we should have an exact birthday — would you mind sharing it (or noting if my estimate was off)?”

The re-prompt is generated by the persona in-character. After the customer responds:

- If they provide an exact DOB: `date_of_birth_is_estimated` = false, `estimation_basis` cleared, `date_of_birth` updated.
- If they decline or ignore: `estimation_last_reprompt_at` = NOW(). No further re-prompt for another 365 days.

Background cleanup

Inngest cron `dob-estimate-reprompt-eligible` runs nightly:

- Finds `customer_memories` rows where any family member has `date_of_birth_is_estimated` = true AND `estimation_recorded_at` < NOW() - 365 days AND (`estimation_last_reprompt_at` IS NULL OR `estimation_last_reprompt_at` < NOW() - 365 days).
- Sets `awaiting_dob_reprompt`: true flag on the `customer_memories` row.
- On the customer’s next conversation turn, the persona prompt augmentation logic (per §9.7) reads this flag and includes the re-prompt instruction.
- The flag clears once a re-prompt has been issued.

A customer who hasn’t conversed with the tenant in 90+ days is not re-prompted — no point waking up a dormant relationship to ask about a birthday.

Schema

```
ALTER TABLE public.customer_memories
ADD COLUMN awaiting_dob_reprompt BOOLEAN NOT NULL DEFAULT FALSE;
```

The estimation tracking fields live inside the `family_composition` JSONB per §11.1; this adds the gate flag for fast read by the conversation handler.

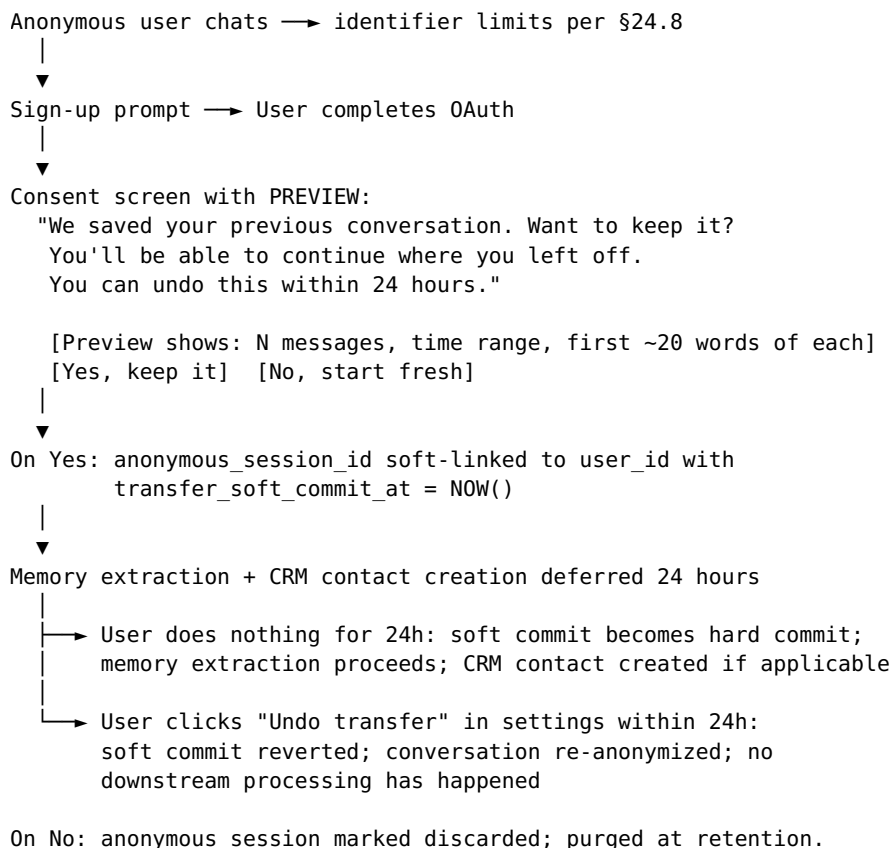
Calls Worth Flagging

- **Suppression in pre-cruise emails is the most important UX guard.** A “celebrating Mike’s 40th!” email when Mike is actually 38 is annoying; when Mike is actually 47, worse.
- **Re-prompt cadence (365 days) is generous.** Re-prompting every 90 days creates conversation friction.
- **The persona delivers the re-prompt, not a system message.** Fits the chat.
- **Estimated DOBs persist through customer deletion grace.** On CCPA deletion, treated same as confirmed DOBs per §25.4a.

11.6 Anonymous-to-Authenticated Transfer

Anonymous customers can chat for a limited number of messages before sign-up is required per §24.8. When they sign up, the platform offers explicit consent to transfer the anonymous session into their authenticated identity. The consent UX has three properties: **preview before accept, distinct from sign-up flow, and 24-hour reversibility.**

Flow



Preview rendering

The consent screen MUST show:

- Total message count being transferred.
- Time span of the conversation (first message → last message).

- The first ~20 words of each message (truncated, not full content).
- A note: “These messages will be available to you and to your trip consultant (if you book through this organization). If anything in these messages was personal or sensitive, you can choose ‘No, start fresh.’”

The preview is the H-3 fix: a user who clicked through a generic “keep your conversation?” prompt without thought now sees what’s about to be associated with their authenticated identity.

24-hour soft commit

After the user clicks “Yes, keep it”:

- `anonymous_sessions.transfer_soft_commit_at = NOW()`,
`anonymous_sessions.transferred_to_user_id = user_id`.
- The session is now associated with the user — they can continue the conversation immediately.
- BUT: downstream processing that creates derived data is DEFERRED for 24 hours.

Deferred-processing rule

For 24 hours after `transfer_soft_commit_at`, the following do NOT execute against the transferred messages:

- Memory extraction (`extractMemory` per §11.2).
- CRM contact creation.
- Pre-cruise email scheduling.
- Persona memory updates beyond what’s needed to keep the conversation flowing.

If the user invokes “Undo transfer” within 24h:

- `anonymous_sessions.transfer_soft_commit_at = NULL`,
`anonymous_sessions.transferred_to_user_id = NULL`.
- The conversation is moved back to anonymous-session state with the user’s authenticated session unaffected. The user can start fresh immediately.
- No derived data needs cleanup — the deferred-processing rule ensured none was created.

After 24h elapses without an undo:

- `anonymous_sessions.transfer_committed_at = NOW()`.
- Inngest job `transfer-finalize` fires, running the deferred processing.
- Subsequent thumbs / feedback / memory updates apply normally.

Where the “Undo” affordance lives

A persistent banner at the top of `/settings/conversations` for the 24h post-transfer window:

“We saved your previous (pre-signup) conversation with [Persona Name] from [DateTime]. [Undo this transfer] (available for [N hours] more)”

After 24h, the banner disappears and the link is replaced with “Transfer made permanent on [DateTime].”

Schema additions

```
ALTER TABLE public.anonymous_sessions
  ADD COLUMN transfer_soft_commit_at TIMESTAMPTZ,
  ADD COLUMN transferred_to_user_id UUID REFERENCES
public.users(id),
  ADD COLUMN transfer_committed_at TIMESTAMPTZ,
  ADD COLUMN transfer_undo_count INTEGER NOT NULL DEFAULT 0;

CREATE INDEX anonymous_sessions_pending_commit_idx
  ON public.anonymous_sessions(transfer_soft_commit_at)
  WHERE transfer_soft_commit_at IS NOT NULL
  AND transfer_committed_at IS NULL;
```

Audit

Every undo writes to audit_log with action = 'session_transfer undone', changes snapshot of message count and time range.

Calls Worth Flagging

- **24h is the right window length for “I clicked through without thinking.”** Longer creates a long tail of pending commits; shorter doesn’t accommodate users who returned next morning to reconsider.
- **Deferred-processing simplifies undo dramatically.** Without it, undo would need to “un-extract memory,” “delete CRM contact,” etc. With it, undo is “set columns to NULL.”
- **The preview is the H-3 fix; the undo is the safety net.** Both are required.
- **transfer_undo_count is signal-not-surfaced.** Logged for future observation if a customer pattern of “consent → undo → re-consent” emerges.

11.7 Memory Audit

All memory changes audit-logged: who changed what when (customer self-edit, AI extraction, agent edit). 7-year retention.

11.8 Memory and CRM

Memory is for the AI’s conversational context. CRM (§12) is for structured records used by agents. They share some data (DOB, dietary, accessibility) but serve different purposes. Updates to one don’t automatically propagate to the other unless explicitly handled in the data flow.

Section 12 — CRM

Customer Relationship Management for travel agents. Multi-traveler contact records, relationship graphs, quote builder, and pipeline.

12.1 Contacts

```

CREATE TABLE public.contacts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id),
  user_id UUID REFERENCES public.users(id), - nullable; not all
  contacts are platform users

  - Identity
  first_name TEXT,
  last_name TEXT,
  middle_name TEXT,
  preferred_name TEXT,
  email TEXT,
  phone TEXT,

  - Demographics
  date_of_birth DATE,
  date_of_birth_is_estimated BOOLEAN DEFAULT FALSE,
  estimation_basis TEXT,
  gender TEXT,
  nationality TEXT,

  - Travel-relevant
  passport_expiry DATE,
  loyalty_programs JSONB,
  dietary_restrictions TEXT,
  accessibility_needs TEXT,

  - Source
  source TEXT, - 'manual', 'ai_extracted', 'signup', 'imported', 'referred'
  source_reference TEXT,

  - Audit
  created_by_user_id UUID REFERENCES public.users(id),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX contacts_tenant_idx ON public.contacts(tenant_id);

CREATE INDEX contacts_email_idx ON public.contacts(tenant_id,
email);

```

```
CREATE INDEX contacts_name_idx ON public.contacts(tenant_id,
last_name, first_name);
```

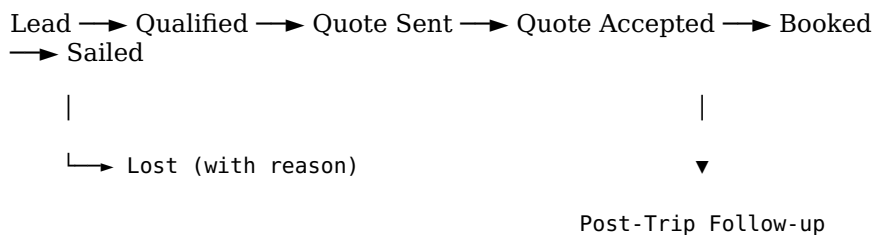
12.2 Contact Relationships

Contacts are linked via a relationship graph. Each link captures a directional relationship type with optional notes.

```
CREATE TABLE public.contact_relationships (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES public.tenants(id),
from_contact_id UUID NOT NULL REFERENCES public.contacts(id)
ON DELETE CASCADE,
to_contact_id UUID NOT NULL REFERENCES public.contacts(id) ON
DELETE CASCADE,
relationship_type TEXT NOT NULL, -
'spouse','partner','parent_of','child_of','sibling','friend','colleague'
notes TEXT,
source TEXT, - 'manual','ai_inferred'
confidence NUMERIC(3,2),
created_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE (tenant_id, from_contact_id, to_contact_id, relationship_type)
);
```

12.3 Pipeline

Default sales pipeline stages (tenant-configurable):



12.4 Quotes

```
CREATE TABLE public.quotes (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES public.tenants(id),
contact_id UUID NOT NULL REFERENCES public.contacts(id),
user_id UUID REFERENCES public.users(id),
```

- Trip details

cruise_line TEXT,

ship_name TEXT,

sailing_date DATE,

duration_nights INTEGER,

cabin_category TEXT,

passenger_count INTEGER,

- Financial

commissionable_fare NUMERIC(12,2),

non_commissionable_total NUMERIC(12,2),

total_amount NUMERIC(12,2),

- Lifecycle

status TEXT CHECK (status IN
('draft', 'sent', 'viewed', 'accepted', 'declined', 'expired', 'converted')),

sent_at TIMESTAMPTZ,

viewed_at TIMESTAMPTZ,

accepted_at TIMESTAMPTZ,

declined_at TIMESTAMPTZ,

declined_reason TEXT,

valid_until TIMESTAMPTZ,

- Conversion

converted_to_booking_id UUID REFERENCES public.bookings(id),

- Display

show_breakdown_to_customer BOOLEAN DEFAULT TRUE,

custom_notes TEXT,

- Audit

created_at TIMESTAMPTZ DEFAULT NOW(),

updated_at TIMESTAMPTZ DEFAULT NOW()

);

Quote PDFs default to FULL breakdown of line items (tenant-configurable). Customer sees what's included and what's optional.

12.5 Commissionable Fare

Commission is computed against the commissionable_fare, not the total. This is a critical distinction: taxes, port fees, gratuities, and some addons typically aren't commissionable.

Line Item Category	Commissionable?
Base cruise fare	Yes
Cabin category upgrade	Yes
Loyalty discount	Reduces commissionable fare
Taxes / port fees	No
Gratuities (pre-paid)	Usually No (varies by line)
Insurance	No
Specialty dining	Varies (line-specific)
Beverage packages	Yes (most lines)
Excursions	Yes (when booked through cruise line)

12.6 Host Booking Fees

Host agencies often charge a per-booking fee deducted from gross commission BEFORE the platform's share is calculated. This means both the platform and the sub-host see proportionally smaller cuts when a host fee applies.

- Host booking fee can be flat, percent, tiered, or none

```
CREATE TABLE public.host_booking_fee_configs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  host_adapter TEXT NOT NULL,
  fee_type TEXT NOT NULL CHECK (fee_type IN
    ('none', 'flat', 'percent', 'tiered')),
  flat_fee_amount NUMERIC(12,2),
  percent_of_commission NUMERIC(5,4),
  tiered_rules JSONB,
  minimum_commission_threshold NUMERIC(12,2),
  effective_from DATE NOT NULL,
  effective_to DATE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE public.tenant_host_fee_overrides (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id),
  host_adapter TEXT NOT NULL,
  - Same fields as host_booking_fee_configs (overrides for this tenant)
  fee_type TEXT NOT NULL,
```



```

flat_fee_amount NUMERIC(12,2),
percent_of_commission NUMERIC(5,4),
tiered_rules JSONB,
minimum_commission_threshold NUMERIC(12,2),
effective_from DATE NOT NULL,
effective_to DATE,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

12.7 Commission Math (Worked Example)

Booking: \$5,000 commissionable fare; host commission rate 15%; host booking fee \$25 flat; sub-host on Pro tier (20/80 split):

Step	Calculation	Amount
Gross commission	$\$5,000 \times 15\%$	\$750.00
Host booking fee	Flat \$25	-\$25.00
Net commission to platform	$\$750 - \25	\$725.00
Platform takes (20%)	$\$725 \times 20\%$	\$145.00
Sub-host retains (80%)	$\$725 \times 80\%$	\$580.00

Section 13 — Host Agency Abstraction Layer

The platform must work without being tightly coupled to any specific host agency. v6 makes the host agency a pluggable adapter — adding a new host means writing an adapter, not rewriting the platform.

13.1 Goals

- Decouple application from host-specific API quirks
- Allow multiple host configurations simultaneously
- Support ‘no host configured’ state gracefully (via fallback email adapter)

13.2 The HostAgencyClient Interface

```

export interface HostAgencyClient {
  readonly adapterId: string;
  readonly displayName: string;
  readonly capabilities: HostCapabilities;
}

```

```

searchInventory(req: InventorySearchRequest, ctx: HostCallContext):
Promise;

submitBooking(req: BookingSubmissionRequest, ctx:
HostCallContext): Promise;

fetchBookingStatus(ref: string, ctx: HostCallContext): Promise;

cancelBooking(ref: string, req: CancellationRequest, ctx:
HostCallContext): Promise;

modifyBooking(ref: string, req: ModificationRequest, ctx:
HostCallContext): Promise;

fetchCommissionStatement(req: StatementFetchRequest, ctx:
HostCallContext): Promise;

recordCommissionPayment(req: CommissionPaymentRequest, ctx:
HostCallContext): Promise;

healthCheck(): Promise;

describeCapabilities(): HostCapabilities;
}

export interface HostCapabilities {
supports_inventory_search: boolean;
supports_real_time_booking: boolean;
supports_modification: boolean;
supports_cancellation: boolean;
supports_commission_api: boolean;
booking_types: BookingType[];
cruise_lines_supported: string[];
commission_currency: string;
payment_lag_days_typical: number;
}

```

13.3 Adapter Registry

```

CREATE TABLE public.host_adapters (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
adapter_id TEXT UNIQUE NOT NULL,
display_name TEXT NOT NULL,
implementation_path TEXT NOT NULL,
config JSONB NOT NULL,
capabilities JSONB NOT NULL,

```

```

is_active BOOLEAN DEFAULT TRUE,

is_default BOOLEAN DEFAULT FALSE,

health_check_status TEXT CHECK (health_check_status IN
('healthy','degraded','unhealthy','unknown')),

health_check_last_at TIMESTAMPTZ,

health_check_message TEXT,

created_at TIMESTAMPTZ DEFAULT NOW()

);

CREATE UNIQUE INDEX host_adapters_default_idx
ON public.host_adapters(is_default) WHERE is_default = TRUE;

```

13.4 Tenant Host Configuration

```

CREATE TABLE public.tenant_host_configs (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON
DELETE RESTRICT,

adapter_id TEXT NOT NULL REFERENCES
public.host_adapters(adapter_id),

credentials JSONB NOT NULL, - encrypted at rest (AES-256-GCM,
KMS key in env)

credential_status TEXT NOT NULL DEFAULT 'pending'

CHECK (credential_status IN
('pending','verified','rejected','expired','revoked')),

credential_verified_at TIMESTAMPTZ,

is_active BOOLEAN DEFAULT TRUE,

UNIQUE (tenant_id, adapter_id)

);

```

13.5 Credential Storage Pattern

Tenant host credentials are encrypted at the application layer before being written to the DB. The encryption uses AES-256-GCM with a per-record IV. This protects against DB-level breach: an attacker who somehow gets read access to the DB sees only ciphertext.

Per-tenant credentials cannot be in environment variables (would require deployment per onboarded tenant). DB storage with app-layer encryption is the right pattern for this cardinality.

13.5.1 Key storage and rotation

The encryption key set is held in environment variables:

- `APP_ENCRYPTION_KEY_CURRENT` — 256-bit key (base64 encoded), used for new writes.
- `APP_ENCRYPTION_KEY_PREVIOUS` — set during key rotation; used for decrypting records written under the previous key. Read-path only.
- `APP_ENCRYPTION_KEY_ID_CURRENT` — short identifier (e.g., v3) stamped onto every ciphertext so the decryption path knows which key to use.

Rotation is annual per §28.20. The rotation flow:

1. Generate new key. Set `APP_ENCRYPTION_KEY_PREVIOUS` = <current>, `APP_ENCRYPTION_KEY_CURRENT` = <new>, `APP_ENCRYPTION_KEY_ID_CURRENT` = <new id>. Deploy.
2. All new writes use the new key.
3. Decryption path: tries `CURRENT` first; on key-id mismatch, falls back to `PREVIOUS`.
4. A background job (Inngest, re-encrypt-old-records) re-reads and re-writes records still ciphered under the previous key. Job runs until zero records remain at the old key id.
5. Remove `APP_ENCRYPTION_KEY_PREVIOUS` from environment. Rotation complete.

13.5.2 Disaster recovery — the failure modes

The encryption keys are operator-managed secrets. Three failure modes:

1. **Both keys lost** (env vars deleted in Vercel, no offsite backup): every encrypted record becomes unrecoverable. The data is intact in the database but cannot be decrypted. Effect: every tenant's host adapter credentials, every tenant's Resend API key, every tenant's encrypted secret is dead. Recovery requires every tenant to re-enter their credentials.
2. **Decryption failure on a specific record** (corrupt ciphertext, key-id mismatch with no matching key, GCM auth tag mismatch): a single tenant credential cannot be decrypted. Effect depends on which credential — if it's the tenant's host adapter, bookings for that tenant pause until resolved.
3. **Key rotation gone wrong** (`CURRENT` key incorrect, `PREVIOUS` key removed prematurely, key IDs mismatched): some records decrypt correctly, others don't. The system appears to work for some tenants and not others; diagnosis is harder than (1) or (2).

13.5.3 Disaster recovery — controls

To prevent failure mode (1):

- **Offsite key backup is mandatory.** At setup time and on every rotation, the current and previous keys are stored in a secondary location independent of Vercel env vars. Recommended: 1Password business vault, or AWS Secrets Manager in a separate AWS account, or a hardware-backed offline copy stored physically. The choice is operator preference; the discipline is non-negotiable.
- **Backup verification quarterly.** Operator restores the key set from the backup to a sandbox environment and confirms decryption of a known-good test ciphertext. If the test fails, the backup is unreliable — fix immediately. Logged to `MEMORY.md` per §A.X (see cross-reference at the end of this section).
- **Vercel env var deletion is gated.** Platform admin's Vercel

account does NOT have permission to delete APP_ENCRYPTION_KEY_* env vars without 2FA challenge. (Vercel may or may not support this granularity natively; if not, a runbook step requires the deletion to be performed only after the backup is verified — enforced by operator discipline.)

To handle failure mode (2):

- The decryption wrapper catches decryption failures and returns a structured error (NOT a thrown exception that crashes the request). The error includes the record's key id so investigation is fast.
- Code paths that consume credentials check for the structured-error result and pause the tenant's affected operations rather than silently retrying or returning incorrect data.
- Specifically: if a host adapter's credentials fail to decrypt, the booking submission path returns a tenant-facing error ("Your host adapter credentials need to be re-entered — please visit Settings > Host Integration"). It does NOT silently fall back to the email adapter (per §13.6) — that would mask a security incident.
- Decryption failures are logged to audit_log with action = 'credential.decryption_failed' and a redacted ciphertext hash (NOT the ciphertext itself) for forensic correlation.

To handle failure mode (3):

- A boot-time check (extending §28.19's env validation) confirms that BOTH APP_ENCRYPTION_KEY_CURRENT and APP_ENCRYPTION_KEY_ID_CURRENT are present and that the keys are valid 256-bit base64 strings. If APP_ENCRYPTION_KEY_PREVIOUS is set, it must also be valid.
- The re-encrypt-old-records job has a monitoring metric: credentials_at_previous_key_count. Alert if this metric is non-zero for more than 7 days after a rotation — indicates the job is stuck or rotation is incomplete.

13.5.4 Decryption-failure tenant operations

When a tenant's credential cannot be decrypted, the tenant's state transitions to a partial-degraded mode. The semantics:

- **Reads of historical data unaffected.** Booking history, commission records, conversation messages — all still readable. The affected credential is specifically the host-adapter or external-service credential.
- **New operations requiring the failed credential are blocked.** The tenant sees a clear in-app banner: "Your [host adapter / Resend account / etc.] credentials cannot be loaded. Please re-enter them in Settings to resume [bookings / outbound email / etc.]"
- **Tenant-facing operation that doesn't need the failed credential proceeds normally.** A tenant whose host-adapter credential failed can still send and receive emails, view CRM, etc.
- **Platform admin alerted on first occurrence per tenant.** Not on every retry — first decryption failure for a given credential per tenant per 24 hours opens a high-priority issue.

13.5.5 Calls Worth Flagging

- **Offsite key backup is the single point of failure prevention.** The cost is low (a few minutes to copy keys to 1Password); the cost

of skipping it is unrecoverable data. Make backup verification a Phase-0 launch gate.

- **The decryption-failure tenant-facing error is deliberately specific.** A generic “something went wrong” lets the tenant click Retry indefinitely; a specific “credentials need to be re-entered” gets the tenant to the right place quickly.
- **Re-encryption job timing matters during rotation.** The job re-writes every encrypted record. For a tenant with many encrypted records (multiple host adapters, many API keys), the re-write produces audit log noise. Acceptable, but expect a spike in `credential.rotation_reencrypt` audit rows during the days following rotation.
- **The `APP_ENCRYPTION_KEY_*` env vars are NOT the same secret class as the Supabase service role key.** Service role key rotation per §28.20 is “on compromise only”; encryption keys rotate annually. Different threat models, different cadences. Don’t conflate.

This subsection cross-references §28.13 (env var definitions) and §28.20 (rotation policy).

13.6 The Fallback Email Adapter

Always-available adapter. Implements `HostAgencyClient` but does no integration — emails structured booking details to a configured destination. Used when:

- Platform is in early operation without a host integration
- A BYO-host tenant hasn’t configured their own host adapter
- A real adapter is degraded or down

Returns synthetic `provider_booking_ref` like ‘EMAIL-{tenant_id}-{timestamp}’. Tenant admin manually updates with real reference once host confirms.

13.7 Adapter Selection Flow

`submitBooking(tenant, booking)`

|

▼

Tenant has verified host config? → Yes → use tenant’s adapter

| No

▼

Tenant is sub-host? → Yes → use platform default + sub-host credentials

| No

▼

Tenant is platform? → Yes → use platform default

| No